CS405 Project 1 — Transformations, Projection & Curves from Paintings

Painting: Şeker Ahmed Paşa — *Kavunlu Natürmort* (Still Life with Melon), Sakıp Sabancı Museum

Joshua · CS405 Introduction to Computer Graphics, Fall 2026 · Repository: github.com/AbstractJosh/CS405-Project

1 The painting and the scene

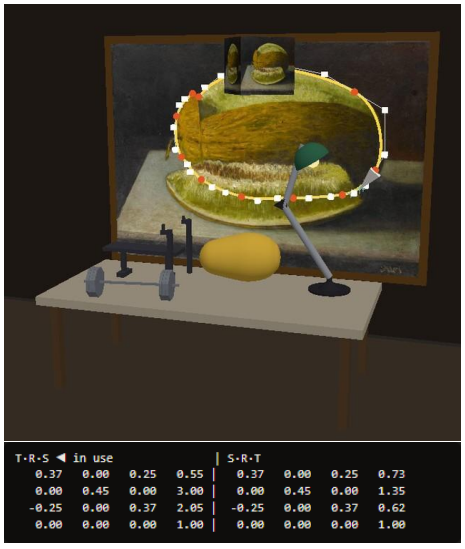


Kavunlu Natürmort (720 × 502 px)

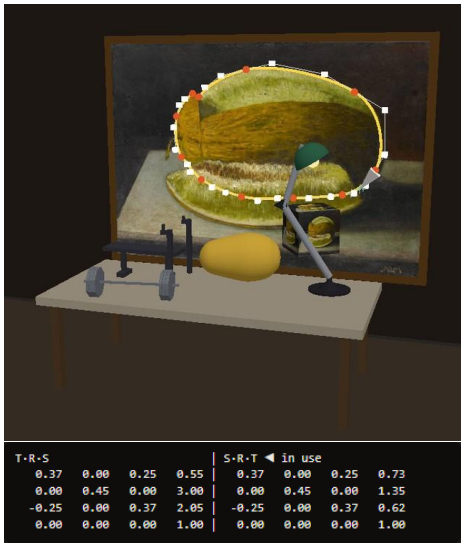
I chose *Kavunlu Natürmort* because the brief lists it for **strong organic contours**: the large melon is one big closed outline, a natural target for Part D, and it gave the scene a theme — a still life on a table in front of the still life on the wall. The painting hangs on the back wall as a textured quad (3.6 × 2.51 units, the image's 720 : 502 aspect). On the table stand a weight bench, a barbell, the Part A box and an animated desk lamp built from five parts — all seven OBJ models come from XiaoShua3D, an earlier project of mine (Appendix C) — plus a melon generated from the traced curve (the bonus). It is one WebGPU application with WGSL shaders; all matrix, camera, scene-graph and curve maths is in the repository, with no maths or 3D library.

2 Part A — Transformations

`math/mat4.js` implements identity, multiply, translate, rotateX/Y/Z, scale, transpose and the rigid inverse ($[R \mid t]^{-1} = [R^T \mid -R^T t]$). Matrices are **column-major** `Float32Array(16)` used with **column vectors** ($p' = M \cdot p$) and uploaded unchanged. Every object is placed with them, e.g. the bench with $T(-0.8, 0.92, -0.2) \cdot R_y(\pi/2) \cdot S(0.8)$. The panel drives the textured box and always shows both products, marking the one in use with ◀ (Figure 1).



M = T·R·S (in use)



M = S·R·T (in use)

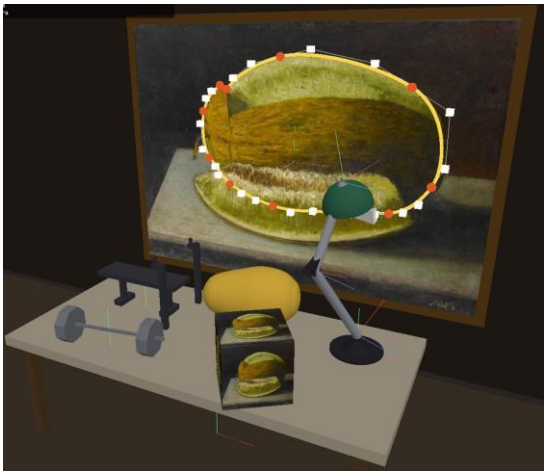
Figure 1. The same sliders — t = (0.55, 3.00, 2.05), R = $R_y(0.6)$, s = 0.45 — under T·R·S (left) and S·R·T (right).

The right-most matrix acts first. **T·R·S** scales and rotates the box about its own centre, then moves it to t, so the sliders mean "put the box here". **S·R·T** translates first, so the move itself is then rotated about the world y-axis and shrunk by s: the last column becomes $S \cdot R \cdot t = 0.45 \cdot R_y(0.6) \cdot (0.55, 3.00, 2.05) = (0.73, 1.35, 0.62)$, as the panel shows. The 3 × 3 block is identical ($0.37 = 0.45 \cos 0.6$, $0.25 = 0.45 \sin 0.6$) because a uniform scale commutes with a rotation; with a non-uniform scale $R \cdot S \neq S \cdot R$ as well. With the default t the S·R·T box lands at (0.73, 0.52, 0.62), under the table top, which is why it seems to vanish. **All other code uses M = T·R·S.**

3 Part B — Scene graph

In `scene/node.js` a node has a local matrix, a parent, children and a world matrix; `update()` walks the tree depth-first setting world = world(parent) · local, so a parent is ready before its children. (The x / y / z sliders under "inspect node" add a translation on top: world = world(parent) · T(offset) · local.) The tree has 27 nodes on 7 levels; the panel lists the 12 frame nodes. The desk lamp is the animated chain, with t in seconds:

$$\text{arm 1} = T(0, 0.127, 0) \cdot R_y(0.6t) \cdot R_z(0.5 + 0.2 \sin 1.3t)$$
$$\text{arm 2} = T(0, 0.8, 0) \cdot R_z(-1.2 + 0.35 \sin(0.9t + 1))$$
$$\text{lamp head} = T(0, 0.6, 0) \cdot R_z(0.7 + 0.25 \sin 1.7t)$$



```
root
├─ wall, floor                  (geometry)
├─ painting ── canvas, frame
│  └─ follower                  (arrow, Part D)
├─ table ── table top, legs ×4, melon
│  ├─ bench
│  ├─ barbell
│  └─ lamp base ── lamp foot
│     └─ arm 1 ── rod 1, elbow
│        └─ arm 2 ── rod 2
│           └─ lamp head ── shade
│              └─ bulb
└─ part A                       (Part A box)
```

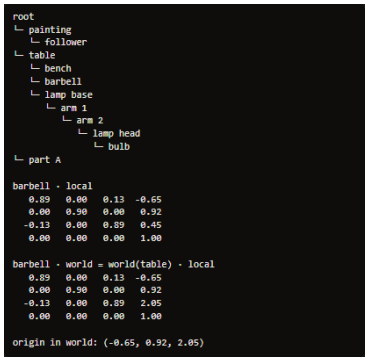
Figure 2. The hierarchy in motion with node axes drawn (debug mode), and the tree; "—" lists a node's geometry children.

World matrix of a leaf, by hand: the barbell (root → table → barbell)

The root is the identity, world(table) = T(0, 0, 1.6) and local = T(−0.65, 0.92, 0.45)·R_y(0.15)·S(0.9), with 0.9 cos 0.15 = 0.8899 and 0.9 sin 0.15 = 0.1345:

$$\text{world(barbell)} = T(0,0,1.6)\cdot\text{local} = \begin{bmatrix} 0.8899 & 0 & 0.1345 & -0.65 \\ 0 & 0.9 & 0 & 0.92 \\ -0.1345 & 0 & 0.8899 & 2.05 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

A pure translation only adds (0, 0, 1.6) to the last column (0.45 + 1.6 = 2.05), so the barbell's base centre is at (−0.65, 0.92, 2.05), exactly on the table top (y = 0.88 + 0.04 = 0.92). This matches the app's panel (right).



```
root
└ painting
   └ follower
└ table
   └ bench
   └ barbell
   └ lamp base
      └ arm 1
         └ arm 2
            └ lamp head
               └ bulb
└ part A

barbell · local
   0.89   0.00   0.13  -0.65
   0.00   0.90   0.00   0.92
  -0.13   0.00   0.89   0.45
   0.00   0.00   0.00   1.00

barbell · world = world(table) · local
   0.89   0.00   0.13  -0.65
   0.00   0.90   0.00   0.92
  -0.13   0.00   0.89   2.05
   0.00   0.00   0.00   1.00

origin in world: (-0.65, 0.92, 2.05)
```

Panel: barbell local / world

4 Part C — Camera and projection

View matrix. The orbit camera stores a target, azimuth y, elevation e and distance d: eye = target + d(cos e sin y, sin e, cos e cos y). `lookAt` builds the camera basis with **two cross products**:

$$w = \text{normalize}(\text{eye} - \text{target}) \qquad u = \text{normalize}(\text{up} \times w) \qquad v = w \times u$$

w points backwards (the camera looks down −w), and v is already unit length because w ⊥ u. The view matrix inverts the camera frame [u v w | eye]; its 3 × 3 part is orthonormal, so the inverse is the transpose:

$$V = [\ u^T\ \ -u\cdot\text{eye}\ ;\ v^T\ \ -v\cdot\text{eye}\ ;\ w^T\ \ -w\cdot\text{eye}\ ;\ 0\ 0\ 0\ 1\ ]$$

Check (Figure 3): y = 0.3, e = 0.2, d = 7.5 give eye = (2.17, 2.99, 7.92), w = (0.29, 0.20, 0.94), u = (0.96, 0, −0.30) and −w·eye = −8.64, the rows the app prints. **Degenerate case:** if up ∥ w (the "Top view" button), up × w ≈ 0, so `lookAt` substitutes the world axis least aligned with w (smallest $|w_i|$), which is never parallel to w.

Projection. With $f_y = 1/\tan(\text{fovY}/2)$: $x_{clip} = (f_y/\text{aspect})x$, $y_{clip} = f_y \cdot y$ and $w_{clip} = -z$. Let $z_{clip} = Az + B$. WebGPU needs $z_{ndc} = z_{clip}/w_{clip}$ in **[0, 1]** (not OpenGL's [−1, 1]), so z = −n must map to 0 and z = −f to 1:

$$(-A\,n + B)\,/\,n = 0 \Rightarrow B = A\,n \qquad (-A\,f + A\,n)\,/\,f = 1 \Rightarrow A = f\,/\,(n - f),\ \ B = n\,f\,/\,(n - f)$$

For n = 0.5, f = 30, fov 50°, aspect 1.16: f_y = 2.145, f_y/aspect = 1.849, A = −1.017, B = −0.508 — the P in Figure 3. `orthographic` is linear, $z_{ndc} = (z + n)/(n - f)$, with half-height d·tan(fov/2), so switching projection keeps the framing.

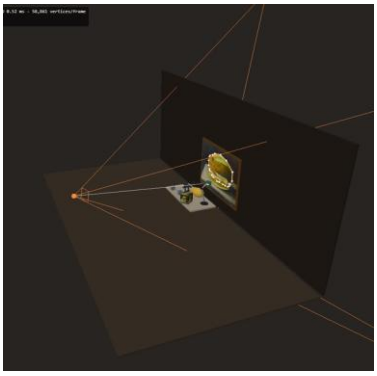
Near, far and depth precision. Perspective depth $z_{ndc} = f(d - n)/(d(f - n))$ is hyperbolic in the distance d: half the [0, 1] range is used before $d = 2nf/(n + f)$, which is 0.98 for my **n = 0.5, f = 30**, while the set is 5–10 units away. n = 0.5 practically never cuts into the set (the camera stays at least 2 units from its target), and raising n helps precision far more than lowering f; f = 30 covers the maximum orbit distance of 25. Since $dz_{ndc}/dd = nf/(d^2(f - n))$, a `depth24plus` buffer (step $\varepsilon \approx 6\cdot10^{-8}$) resolves $\Delta d \approx \varepsilon\cdot d^2(f - n)/(nf) \approx 7.5\cdot10^{-6}$ at the painting (d ≈ 8). The smallest gap in the scene is 0.01 (canvas to frame); even n = 0.05, f = 100 only gives $\Delta d \approx 7.6\cdot10^{-5}$, 130 × smaller, so no z-fighting appears (it would need $n \approx 4\cdot10^{-4}$). Orthographic depth is linear, so there n and f only clip; pulling f in to 6 clips the painting, wall and table, which are 7–8 units away.



perspective



orthographic



inspection camera: main camera + frustum

```
eye     (2.17, 2.99, 7.92)
target  (0.00, 1.50, 0.90)
up      (0.00, 1.00, 0.00)
fov y   50°   aspect 1.16
near    0.5   far 30
proj    perspective

V = lookAt(eye, target, up)
   0.96   0.00  -0.30   0.27
  -0.06   0.98  -0.19  -1.30
   0.29   0.20   0.94  -8.64
   0.00   0.00   0.00   1.00

P  (depth → z ∈ [0, 1])
   1.849   0.000   0.000   0.000
   0.000   2.145   0.000   0.000
   0.000   0.000  -1.017  -0.508
   0.000   0.000  -1.000   0.000
```

```
eye     (2.17, 2.99, 7.92)
target  (0.00, 1.50, 0.90)
up      (0.00, 1.00, 0.00)
fov y   50°   aspect 1.16
near    0.5   far 30
proj    orthographic

V = lookAt(eye, target, up)
   0.96   0.00  -0.30   0.27
  -0.06   0.98  -0.19  -1.30
   0.29   0.20   0.94  -8.64
   0.00   0.00   0.00   1.00

P  (depth → z ∈ [0, 1])
   0.246   0.000   0.000   0.000
   0.000   0.286   0.000   0.000
   0.000   0.000  -0.034  -0.017
   0.000   0.000   0.000   1.000
```

Figure 3. One camera under both projections, with eye, target, up, fov, near, far, V and P from the panel; and the main camera's frustum seen from the fixed inspection camera.

5 Part D — Curves over the painting

Bézier. A cubic segment is $C(t) = \Sigma\, B_i(t)\cdot P_i$ with the Bernstein basis

$$B_0(t) = (1-t)^3 \qquad B_1(t) = 3t(1-t)^2 \qquad B_2(t) = 3t^2(1-t) \qquad B_3(t) = t^3$$

Proof of the endpoints. At t = 0 every B_i except B_0 has a factor t, and $B_0(0) = 1$, so $C(0) = P_0$; at t = 1 every B_i except B_3 has a factor (1 − t), and $B_3(1) = 1$, so $C(1) = P_3$. Differentiating gives $C'(0) = 3(P_1 - P_0)$ and $C'(1) = 3(P_3 - P_2)$. **De Casteljau** repeats linear interpolation, $P_i^{(k)} = (1-t)P_i^{(k-1)} + t\,P_{i+1}^{(k-1)}$; its levels are drawn for the current t, and a unit test checks it against the Bernstein form.

C^1 at a joint. Segments P and Q are C^0 when $Q_0 = P_3$, and C^1 when also $C'(1) = D'(0)$: $3(P_3 - P_2) = 3(Q_1 - Q_0)$, i.e. $Q_1 = 2P_3 - P_2$ — the handles mirror through the knot. `enforceC1` restores this after every edit; the panel shows a remaining error of $5.6\cdot10^{-17}$. **Catmull–Rom** (second family) passes through every point with tangent $(K_{i+1} - K_{i-1})/2$ at K_i, so it is C^1 by construction.

Tracing. The curve lives in the painting node's local plane in normalised coordinates ((0, 0) bottom-left, (1, 1) top-right); mouse clicks are ray-cast onto that plane, so control points are placed and dragged on the painting itself. I traced the **outline of the large melon** (Figure 4): 10 Bézier segments (31 points) and an 18-segment Catmull–Rom curve (19 points), shown in blue when that curve type is selected. The arrow that rides the curve is turned by $\text{atan2}(C'_y\cdot H, C'_x\cdot W)$, W × H = 3.6 × 2.51, because the normalised axes differ in length. *Export JSON* writes `curve.json` (Appendix A), which the app also loads as its default curve.



Figure 4 (right). The traced outline: round markers are knots, squares are Bézier handles, the grey line is the control polygon, the coloured lines are the de Casteljau levels at t = 0.3, and the white arrow follows the tangent.

6 Part E — Integration, interface and performance

One page with a side panel: a view switch (main / inspection / split), *Reset scene*, debug toggles (node axes, frustum, control polygon, de Casteljau) and one section per part. The code is split into `math/`, `scene/`, `render/` (including the OBJ loader and `revolve`) and `ui/`; `tests/` holds 14 unit tests, and the README documents every control. A readout shows frame time, fps, CPU time, vertices per frame and the GPU.

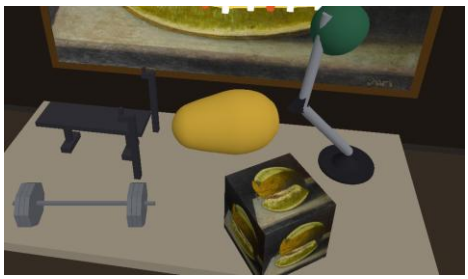
View	Frame time	fps	CPU time / frame (mean)	Vertices / frame
Main camera	6.9–7.0 ms	144	0.56–0.64 ms (0.60 ms)	55,803
Split: main + inspection	6.9 ms	144	0.66–0.75 ms (0.71 ms)	114,664

Measured on my machine: **AMD Radeon RX 9070 XT** (WebGPU reports “amd rdna-4”), Ryzen 7 5700, 144 Hz display, Chrome 156, 1160 × 1000 canvas; five readings after a 2.5 s warm-up. Frame time is pinned to the refresh interval (1/144 Hz = 6.94 ms), so the GPU is never the bottleneck; the app’s own CPU work stays under 1 ms, leaving more than 85 % of each frame free. The split view draws the scene twice: double the vertices, about 0.1 ms more CPU.

7 Bonus — Surface of revolution: a melon from the traced curve

`revolve()` spins the traced curve about its **longest chord** A → B — the two farthest-apart of 201 points sampled along the curve, which for the melon outline are its tips — and revolves the arc between them. With unit direction u, a point p on that arc has axial coordinate a = (p − A)·u and radius r = (p − A) × u, and becomes the ring (a, r cos ϕ, r sin ϕ): 48 rings × 32 sectors. Both ends of the arc lie on the axis, so the surface is closed, and the normal (−dr, da) comes from the curve tangent, so shading is smooth.

Coordinates are scaled by the painting’s size first, so the result has the traced melon’s proportions, 1 : 0.60. The mesh is rebuilt whenever the curve changes, so editing the curve reshapes the melon live. A unit test revolves a semicircle and a closed circle and checks that each gives an exact sphere with outward normals and counter-clockwise triangles.



the revolved melon on the table

8 What went wrong, and how I found it

- **The S·R·T box “disappeared”:** its world-space bounding box (y 0.29–0.74, z 0.31–0.93) showed it under the table top and behind the table — not a bug, but the Part A example.
- **The lamp parts did not fit:** the five OBJ parts were not in one assembled pose. Per-part bounding boxes and profiles showed where each attaches; each part is normalised and placed in its joint frame, and the base’s 0.317-unit post moved the arm-1 pivot from y = 0.06 to 0.317 × 0.4 = 0.127.
- **Normals under non-uniform scale:** the shader transforms normals with the model matrix, exact only for rotation and uniform scale (the exact fix is the inverse-transpose). An early squashed-sphere melon showed it; now only the small curve arrow (0.26 × 0.13 × 0.13) is affected.

Appendix A — Exported curve (curve.json)

The file as exported by the app, with whitespace compacted (the data is identical to `curve.json` in the repository). Coordinates are normalised: (0, 0) is the bottom-left and (1, 1) the top-right of the painting; `knot: true` marks points the Bézier curve passes through, `false` marks handles.


```
{
  "painting": {"artist": "Şeker Ahmed Paşa", "title": "Kavunlu Natürmort (Still Life with Melon)",
               "pixels": [720, 502]},
  "coordinates": "normalised: (0,0) = bottom-left, (1,1) = top-right of the painting",
  "bezier": {"type": "piecewise cubic Bézier", "continuity": "C1", "segments": 10, "points": [
      {"x": 0.3027, "y": 0.7638, "knot": true}, {"x": 0.3235, "y": 0.8057, "knot": false},
      {"x": 0.3768, "y": 0.8587, "knot": false}, {"x": 0.4583, "y": 0.8845, "knot": true},
      {"x": 0.5398, "y": 0.9103, "knot": false}, {"x": 0.7012, "y": 0.8653, "knot": false},
      {"x": 0.7915, "y": 0.7916, "knot": true}, {"x": 0.8818, "y": 0.7179, "knot": false},
      {"x": 0.8862, "y": 0.5302, "knot": false}, {"x": 0.8605, "y": 0.4625, "knot": true},
      {"x": 0.8347, "y": 0.3949, "knot": false}, {"x": 0.7996, "y": 0.359, "knot": false},
      {"x": 0.7644, "y": 0.3489, "knot": true}, {"x": 0.7292, "y": 0.3388, "knot": false},
      {"x": 0.6613, "y": 0.3264, "knot": false}, {"x": 0.6115, "y": 0.3229, "knot": true},
      {"x": 0.5618, "y": 0.3194, "knot": false}, {"x": 0.4969, "y": 0.3023, "knot": false},
      {"x": 0.4436, "y": 0.3174, "knot": true}, {"x": 0.3903, "y": 0.3325, "knot": false},
      {"x": 0.3452, "y": 0.3541, "knot": false}, {"x": 0.3084, "y": 0.3871, "knot": true},
      {"x": 0.2716, "y": 0.4202, "knot": false}, {"x": 0.2622, "y": 0.4553, "knot": false},
      {"x": 0.2417, "y": 0.4881, "knot": true}, {"x": 0.2212, "y": 0.5209, "knot": false},
      {"x": 0.216, "y": 0.6241, "knot": false}, {"x": 0.2265, "y": 0.6745, "knot": true},
      {"x": 0.237, "y": 0.7249, "knot": false}, {"x": 0.2562, "y": 0.754, "knot": false},
      {"x": 0.2821, "y": 0.7777, "knot": true}
  ]},
  "catmullRom": {"type": "uniform Catmull-Rom (end points repeated)", "segments": 18, "points": [
      {"x": 0.2847, "y": 0.7749}, {"x": 0.3481, "y": 0.8394}, {"x": 0.4583, "y": 0.8845},
      {"x": 0.6203, "y": 0.8846}, {"x": 0.7851, "y": 0.7959}, {"x": 0.8874, "y": 0.6391},
      {"x": 0.8876, "y": 0.5024}, {"x": 0.8413, "y": 0.3989}, {"x": 0.793, "y": 0.3565},
      {"x": 0.749, "y": 0.3375}, {"x": 0.6886, "y": 0.3219}, {"x": 0.5993, "y": 0.3072},
      {"x": 0.4944, "y": 0.3152}, {"x": 0.3903, "y": 0.3436}, {"x": 0.3168, "y": 0.388},
      {"x": 0.2693, "y": 0.4445}, {"x": 0.2394, "y": 0.4951}, {"x": 0.2206, "y": 0.6262},
      {"x": 0.2583, "y": 0.7532}
  ]}
}
```


Appendix B — Use of AI tools

Tool: Claude Code (Anthropic), with the Claude Opus 5.5 model, used as a coding and writing assistant throughout the project for the tasks below. The painting choice and the hand-placed curve trace are my own.

Task	What I asked	What I kept
Report	Compile this report from my project: structure it to the brief, write up the text and derivations, check the numbers against the app, capture the screenshots and frame-time readings from the running app, and build the document.	The report as compiled, with its figures and measurements.
Reusing XiaoShua3D	Bring code and models from my earlier project XiaoShua3D (Appendix C) into this project.	The reused code and models.
Application code	Write and change parts of the application code: features, fixes and unit tests.	The code it wrote.
Debugging and explanation	Explain parts of the brief and of my code, and find and fix bugs.	The explanations I used and the fixes.
Housekeeping	Tidy the code and commit and push it to GitHub.	All of it.

Appendix C — Reused material

XiaoShua3D (https://xiaoshua3d.vercel.app) is an earlier project of my own. Some of the code in this project is reused from it, and all of the 3D models come from it: the weight bench (`bench.obj`), the barbell (`barbell.obj`) and the five desk-lamp parts (`lamp_base.obj`, `lamp_lower_arm.obj`, `lamp_arm_joint.obj`, `lamp_upper_arm.obj`, `lamp_head.obj`). The model files are labelled `XiaoShua3D_Model`.